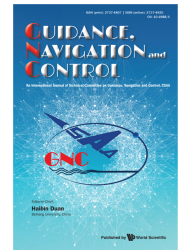




ISSN (print): 2737-4807 | ISSN (online): 2737-4920 | CN: 10-1988/V

Guidance, Navigation and Control

<https://www.worldscientific.com/worldscinet/gnc>

MUTP-LLM: Empowering Multi-UAV Task Planning with Large Language Models

Hongbo Yu ^{*}, Chang Wang ^{*,†}, Yifeng Niu ^{*,†,‡} and Lizhen Wu [†]

^{*}College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, P. R. China

[†]National Key Laboratory of Equipment State and Smart Support, National University of Defense Technology, Changsha 410073, P. R. China

Received 19 May 2025; Revised 18 July 2025; Accepted 26 July 2025; Published 11 September 2025

KEYWORDS

LLMs;
multi-UAV;
task planning

Abstract Although large language models (LLMs) have succeeded in natural language understanding, there are still many challenges in converting natural language instructions into understandable and executable action plans for UAVs while generalizing across different missions. This paper introduces MUTP-LLM, a novel hybrid framework that addresses this gap by integrating the semantic flexibility of LLMs with the formal rigor of traditional planners through a structured, multi-stage architecture. Specifically, our framework first employs an large language model (LLM) to translate ambiguous human commands into a structured symbolic task sequence. A hierarchical planner then generates a high-level plan by allocating and sequencing these tasks. Subsequently, another LLM instance grounds the abstract plan into concrete navigational waypoints. Crucially, a two-stage validation mechanism verifies the plan's logical coherence and physical safety before it is dispatched for execution. Simulation experiments demonstrate that MUTP-LLM achieves superior performance in task success, planning robustness, and safety compared to end-to-end LLM or purely traditional approaches.

1. Introduction

Unmanned Aerial Vehicles (UAVs) are increasingly deployed in coordinated teams, or swarms, to execute complex missions ranging from disaster response to logistics. The key to these operations is efficient and robust task planning, which requires assigning tasks and generating flight paths that satisfy high-level mission goals. However, bridging the ambiguity of human language with the requirements for safe, coordinated execution remains a grand challenge.

Classic solutions for Task and Motion Planning (TAMP)¹ rely on formal representations like PDDL^{2–4} or Temporal logic.^{5,6} These methods offer strong guarantees on plan correctness and are highly interpretable. However, they are fundamentally limited by their reliance on hand-authored world models, which means they cannot reason about concepts outside their pre-defined vocabulary or handle the ambiguity inherent in natural human language. On the other side, learning-based methods, such as semantic parsing^{7–9} or reinforcement learning,¹⁰ have

[‡]Corresponding author.

[§]E-mail addresses: yhb37@nudt.edu.cn (Hongbo Yu); niuyifeng@nudt.edu.cn (Yifeng Niu)

Cite this: Hongbo Yu, Chang Wang, Yifeng Niu and Lizhen Wu, MUTP-LLM: Empowering Multi-UAV Task Planning with Large Language Models, *Guid. Navig. Control*, 5(4), 477–489 (2025), doi: [10.1142/S2737480725500347](https://doi.org/10.1142/S2737480725500347)

© Technical Committee on Guidance, Navigation and Control, CSAA

This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

demonstrated greater flexibility and adaptability. Yet, they often struggle with generalization to novel, out-of-distribution scenarios and can require substantial data or complex reward engineering, limiting their practical applicability in open-world environments.

The recent emergence of Large Language Models (LLMs), such as DeepSeek¹¹ and GPT-4,¹² provides a new paradigm for task planning oriented to understanding complex human intentions. The vast world knowledge and emergent reasoning capabilities allow them to interpret compositional and implicit human intent in a zero-shot or few-shot manner.^{13–15} However, directly using LLMs as end-to-end planners is fraught with risks. They may hallucinate non-existent functions, fail to reason about causality and temporal dependencies,¹⁶ or generate physically infeasible and unsafe action sequences.¹⁷ The lack of inherent closed-loop feedback and formal verification makes such a direct approach unreliable for real-world deployment.^{18,19} This paper addresses this crucial dilemma: how to harness the semantic flexibility of LLMs while preserving the safety, reliability, and stability of traditional planning methods?

This paper introduces MUTP-LLM, a novel hybrid and hierarchical framework that synergizes the strengths of LLMs and traditional planners. MUTP-LLM employs an LLM first as an intent interpreter, translating unstructured, nuanced natural language commands into a formal task structure that a rule-based planner can process for macroscopic mission assignment. Subsequently, it leverages the LLM to convert abstract subtasks into navigational waypoints. This approach is augmented by a two-tiered validation loop. At the strategic level, it validates the logical coherence of the task plan itself. At the grounding level, it filters the generated waypoints for safety and feasibility. This ensures that the final plan is not only contextually relevant but also logically sound and physically executable. MUTP-LLM effectively utilizes LLMs for their superior semantic understanding and reasoning capabilities while delegating structured planning and final action execution to reliable, grounded components. The main contributions are as follows:

- We propose MUTP-LLM, a novel framework that synergizes a traditional hierarchical planner with a Large Language Model, combining the reliability of formal methods with the generalization capability of LLMs for multi-UAV task planning.
- Our framework employs an LLM to first parse complex natural language commands into structured tasks for an HTN planner, and then to convert the decomposed subtasks into a sequence of strategic waypoints,

effectively bridging high-level human intent and low-level motion planning.

- We introduce a two-stage validation framework. A strategic-level validator first ensures the logical soundness of the HTN-generated task plan. Subsequently, a lightweight grounding-level checker verifies the safety of the LLM-generated waypoints, thereby ensuring reliability from high-level logic to final motion.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formally defines the problem. Section 4 details the proposed method. Sections 5 and 6 present the experimental setup and results. Finally, Sec. 7 concludes the paper and discusses future work.

2. Related Works

2.1. Task planning for multi-UAV

Multi-UAV formation is essentially a multi-agent system whose task planning includes several aspects, such as task assignment and path planning. Overall, task planning methods for multi-UAV can be categorized into rule-based and learning-based. A rule-based planning approach provides a structured and easily understandable framework for multi-UAV actions in different situations. One well-known example of rule-based planning is the Hierarchical Task Network (HTN), whose planning process starts by decomposing the initial task network and continues until all compound tasks are decomposed. HTN cuts down on the classical planning search space by relying on a given hierarchical library of domain control knowledge. On this basis, the HATP²⁰ extends the traditional HTN planning domain representation and semantics by making them more suitable for roboticists and treating agents as “first-class” entities in the language. Höller *et al.*²¹ propose an extension to PDDL, the description language used in non-hierarchical planning, to the needs of hierarchical planning systems. While powerful in their formal guarantees, the core limitation of these methods is their semantic rigidity. They operate on a pre-defined symbolic vocabulary, making them unable to interpret ambiguous natural language or generalize to tasks not explicitly encoded in their domain model.

Learning-based approaches have been introduced to improve performance and adaptability. For instance, deep reinforcement learning has been used to create scalable and generalizable task learning frameworks by collecting experience across multiple robots and tasks.²² However, such methods often face challenges with overfitting, poor performance on long-tail scenarios, and

require significant data for training. Ultimately, both classical and learning paradigms struggle to bridge the semantic gap between nuanced, high-level human intent and low-level, executable machine commands. This fundamental challenge motivates the integration of Large Language Models, which possess unprecedented capabilities in natural language understanding and common-sense reasoning.

2.2. Task planning with large-language models

The application of LLMs to robot planning has rapidly evolved, demonstrating remarkable zero-shot generalization in open-world scenarios. However, the paradigm of how to best integrate LLMs is still an active area of research, with different approaches exhibiting distinct trade-offs between flexibility and reliability.

One common paradigm uses LLMs to translate natural language into formal planning languages like PDDL.^{23–25} This leverages a classical planner for its soundness guarantees, but this projection onto a rigid symbolic representation can lead to a context collapse, where subtle but critical information from the original command is lost. In contrast, our MUTP-LLM framework uses a hierarchical translation, where the LLM first interprets intent into a high-level task structure for an HTN planner, preserving the macroscopic mission logic and avoiding the premature flattening of context. Another prominent approach treats LLMs as end-to-end code generators that directly produce executable scripts.^{26,27} While highly flexible, this method is inherently unreliable for robotics, as LLMs may hallucinate invalid actions or produce physically unsafe plans.¹⁶ To mitigate this, hybrid systems have been developed to ground LLM outputs. For example, SayCan¹⁴ cross-references LLM suggestions with learnable value functions. While effective at grounding, this approach requires significant prior effort to train value functions for all skills and does not utilize a structured, symbolic planner for long-horizon reasoning. Code as Policies²⁷ and ProgPrompt²⁸ treat the LLM as a direct generator of programs or API call sequences. These methods improve safety by restricting the LLM to a predefined function library. However, they typically place the full burden of complex, step-by-step logical and temporal reasoning entirely on the LLM.

MUTP-LLM proposes an architecture to address these challenges. We offload the high-level task decomposition and sequencing to a deterministic HTN planner. We then use the LLM first as a high-level intent interpreter and subsequently as a grounded action generator. Furthermore, our framework incorporates a validation module to ensure the generated plan remains consistent with the

original mission goal. This verification step, largely absent in prior work, adds a crucial layer of stability and accuracy, effectively synthesizing the semantic flexibility of LLMs with the formal rigor required for robust multi-UAV operations.

3. Problem Formulation

In this paper, we address the problem of generating a coordinated multi-UAV task and motion plan from a high-level natural language command. We formally define the problem as finding a valid plan Π that accomplishes the task goal specified by a natural language instruction I .

Let $\mathcal{U} = \{u_1, u_2, \dots, u_n\}$ be a set of n heterogeneous UAVs. Each UAV $u_i \in \mathcal{U}$ is characterized by a set of capabilities u_i , such as **reconnaissance**, **strike**, or **transport**, and its state at time t is denoted by $s_i(t) \in S_i$, which includes its position and task status.

The natural language instruction I expresses a high-level mission objective, which the LLM can decompose into a sequence of abstract subtasks $\mathcal{T} = (\tau_1, \tau_2, \dots, \tau_K)$, such as **TargetSearch**, **StrikeExecute**, or **DamageAssessment**. The environment contains a set of static obstacles $\mathcal{O}$ and a set of mission-relevant locations or targets $\mathcal{G} = \{G_1, G_2, \dots, G_M\}$.

The overall task planning problem is to generate a plan $\Pi = \{\pi_1, \pi_2, \dots, \pi_n\}$, where π_i is the individual plan for UAV u_i . Each individual plan π_i is a continuous, collision-free trajectory, which is generated by a motion planner based on a sequence of waypoints.

It is noted that the valid plan Π must respect the causal dependencies between subtasks. And the subtask assigned to UAV u_i must be consistent with its capabilities C_i . Concurrently, all UAV trajectories must be collision-free to obstacles $\mathcal{O}$ and inter-UAV collisions, and must adhere to the dynamic constraints of the UAVs.

The objective is to find such a plan Π that is not only valid but also efficient, often implicitly optimizing for metrics like mission completion time or resource consumption. Our framework, MUTP-LLM, is designed to systematically solve this problem by decomposing it into semantic interpretation, hierarchical task assignment, and grounded action generation stages.

4. Method

We aim to address the challenge of multi-scenario task planning for multi-UAV in a large-scale environment based on natural language instructions by embedding LLMs into a structured planning process. The architecture of our framework, MUTP-LLM, is illustrated in Fig. 1.

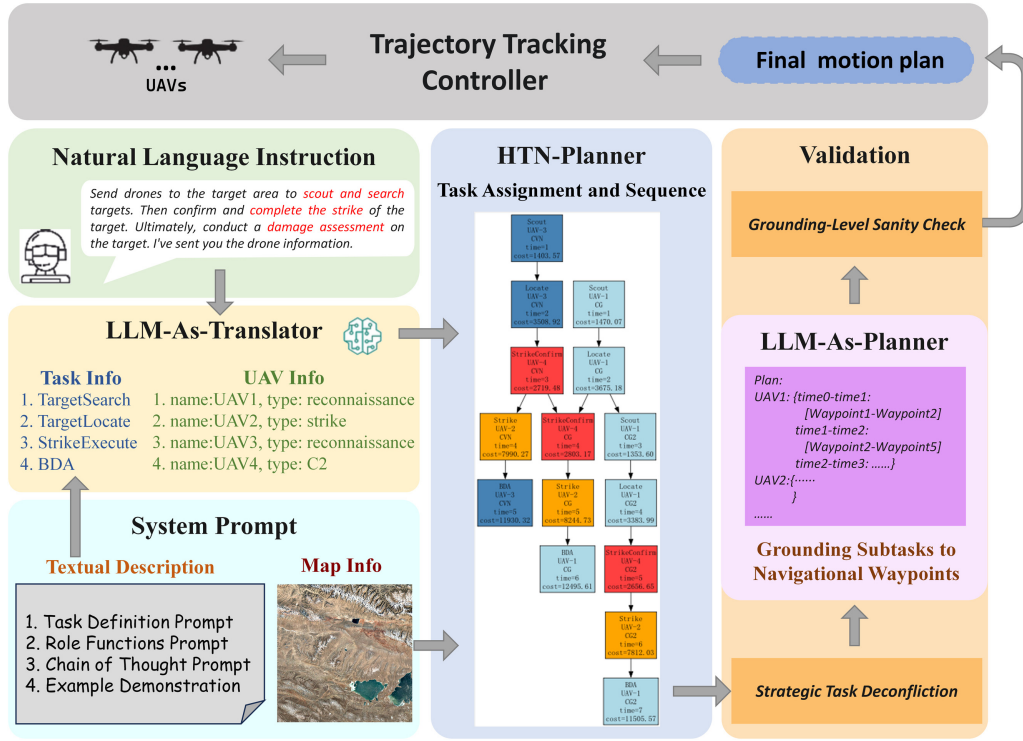


Fig. 1. MUTP-LLM Overview. MUTP-LLM employs a hierarchical approach where an LLM-as-Translator first interprets human intent into structured tasks. A Basic Task-Planner handles symbolic task assignment and sequencing, which is then grounded into navigational waypoints by an LLM-as-Planner. A two-stage validation process ensures the final motion plan is both strategically coherent and physically safe before execution.

The process begins with a high-level natural language instruction from a human operator, which is first processed by the LLM-as-Translator into a structured task definition. This output is then passed to Hierarchical Task Network (HTN), which performs macroscopic task assignment and sequencing for the UAVs. The output from the HTN is first passed to our strategic-level validator, which resolves logical and resource conflicts in the task plan. Once validated, the LLM-as-Planner converts these abstract subtasks into a sequence of navigational waypoints. Finally, these waypoints undergo a grounding-level sanity check for safety and plausibility before being dispatched to the motion planning module, which computes the final executable trajectories. This hierarchical, validation-centric approach ensures reliability. Once a fully verified sequence is formed, it constitutes the final motion plan for execution by the UAVs. This hierarchical, validation-centric approach ensures that complex commands are translated into safe and reliable actions.

4.1. Hierarchical task network (HTN) planner

The robustness and adaptiveness of the Hierarchical Task Network (HTN) demonstrate that basic task planners are capable of successfully task planning for many scenarios.

This provides an intuitive methodology for specifying high-level instructions on how robots and agents should perform tasks while giving the planner enough flexibility to choose the lower-level steps and their ordering.²⁰ HTN takes an initial network of tasks as input, reduces the search space of traditional planning by a given knowledge base, and uses task decomposition methods to break down the entire problem into fine-grained execution actions by individual UAVs. The core strength of HTN lies in its hierarchical structural design, which enables it to handle and respond to multiple complex tasks effectively. The method is summarized in Algorithm 1. By decomposing complex tasks into multiple subtasks and optimizing and coordinating them at different levels, HTN is able to ensure logical and efficient task execution. This hierarchical approach improves the flexibility of task planning and enables it to adapt to changing environmental conditions and task requirements in parallel.

On the other hand, HTN is competent in integrating and utilizing the existing knowledge base effectively. In this manner, HTN can avoid repeating past errors during the planning process and reduce unnecessary explorations and attempts, thus significantly reducing the search space. This intelligent error avoidance mechanism improves the quality of task planning and dramatically

Algorithm 1 HTN Algorithm

```

1: procedure HTN(Tasks, Agents)
2:   Hierarchical_Structure  $\leftarrow$  Initialize_Hierarchy(Tasks)
3:   for all level  $\in$  Hierarchical_Structure do
4:     Subtasks  $\leftarrow$  level.Get_Subtasks()
5:     Avail_Agents  $\leftarrow$  Agents.Get_Avail_Agents(level)
6:     for all task  $\in$  Subtasks do
7:       Best_Agent  $\leftarrow$  Select_Agent(task, Avail_Agents)
8:       if Best_Agent  $\neq$  null then
9:         Best_Agent.Assign_Task(task)
10:        Available_Agents.Remove(Best_Agent)
11:       else
12:        Subtasks_Of_Task  $\leftarrow$  Decompose_Task(task)
13:        level.Add_Subtasks(Subtasks_Of_Task)
14:       end if
15:     end for
16:   end for
17:   for all agent  $\in$  Agents do
18:     Plan  $\leftarrow$  Generate_Plan(agent.Assigned_Tasks())
19:     agent.Set_Plan(Plan)
20:   end for
21:   for all agent  $\in$  Agents do
22:     Execute_Plan(agent)
23:     Monitor_and_Adjust(agent)
24:   end for
25: end procedure

```

enhances the overall computational efficiency. In addition, HTN's task planning algorithm is optimized to reduce the consumption of computational resources while ensuring the effectiveness of task execution. It excludes unreasonable planning solutions through intelligent pruning techniques and retains only the most promising candidates for in-depth analysis. This design enables HTN to maintain efficient and stable performance when dealing with complex tasks.

Mathematically, the HTN planner acts as a function f_{HTN} that takes the abstract subtask sequence $\mathcal{T}$ and the set of available UAVs $\mathcal{U}$ as input. Its output is an assigned task plan H_{assigned} , which is a mapping of subtasks to specific UAVs:

$$H_{\text{assigned}} = f_{\text{HTN}}(\mathcal{T}, \mathcal{U}). \quad (1)$$

This process, detailed in Algorithm 1, implicitly solves an optimization problem to minimize a cost function (e.g. total mission time or energy consumption) subject to the capability constraints C_i of each UAV $u_i \in \mathcal{U}$.

The above algorithms are widely used in the field of UAV navigation and control. UAVs are able to efficiently plan optimal or sub-optimal flight paths while maintaining precise trajectory tracking capabilities, ensuring efficient task completion and operational safety. In addition, the stability and robustness of these algorithms have been

verified in a large number of experiments and practical operations.

4.2. LLM as translator

The LLM-as-Translator module is the entry point of our framework, responsible for parsing the unstructured natural language instruction from the user into a formal representation that the downstream planner can process.

Formally, we model the LLM-as-Translator as a function f_{trans} that maps the natural language instruction I and the system prompt $\mathcal{P}_{\text{trans}}$ (containing UAV capabilities and environmental information) to an ordered sequence of abstract subtasks $\mathcal{T}$:

$$\mathcal{T} = f_{\text{trans}}(I, \mathcal{P}). \quad (2)$$

This function is implemented by querying a large language model with the carefully designed prompt shown in Table 1, which guides the LLM to perform semantic parsing and task decomposition.

The advantage of this conversion process is that it significantly breaks through the limitations of traditional semantic understanding, which often relies on rigid, predefined semantic spaces. By leveraging the powerful conversational capabilities of LLMs, our framework adeptly handles scenarios that are intractable for purely formal planners. Furthermore, it can decompose complex conditional instructions such as "If UAV2 encounters an emergency situation, redirect UAV3 to the task zone",

Table 1. System prompt for LLM-as-Translator.

Role Definition
You are a Mission Command Parser. Your primary goal is to interpret unstructured natural language instructions, decompose them into a sequence of structured subtasks, and assign appropriate UAVs based on their capabilities and a set of operational rules.
World Model and Information
1: UAV Capabilities: A list of available UAVs and their functionalities. Each UAV type is limited to 2 units. (e.g. UAV_Recon: Specialized for reconnaissance tasks.)
2: Task-to-Capability Mapping: (e.g. recon (e.g. "scout", "search", "monitor") requires reconnaissance capability.)
3: Target List: A list of known targets and points of interest with their names and coordinates. (e.g. Command Center: (3396, 1835),...)
Task Parsing and Assignment Rules
1: Priority Levels: Infer task priority from keywords. (e.g. "immediately", "urgent" $\rightarrow$ High; "important" $\rightarrow$ Medium; "confirm" $\rightarrow$ Low).
2: UAV Selection Logic: Select the nearest available UAV that meets the task's capability requirement.
3: Output Verbosity: For assignments involving two UAVs of the same type, append suffixes 'A' and 'B' to their IDs. (e.g. UAV_Strike-A, UAV_Strike-B)

parsing the sentence into a primary task and a contingency rule. This ability to interpret non-standardized human language provides more affluent and robust inputs to the mission planner, which helps to generate more optimized and efficient UAV action strategies. Figure 2 shows the result of subtask assignment and the execution flow obtained by HTN after acquiring the human natural language instructions translated by LLM-as-Translator.

4.3. LLM as planner

The LLM-as-Planner module is tasked with the critical step of translating the validated, high-level task plan H_{valid} from the strategic planner into a sequence of concrete, spatial targets known as navigational waypoints. Its purpose is to bridge the gap between abstract symbolic planning and grounded motion execution.

We formalize this component as a function f_{plan} that maps the assigned plan Π_{assigned} to the final, executable multi-UAV plan $\Pi = \{\pi_1, \dots, \pi_n\}$:

$$W = f_{\text{plan}}(H_{\text{valid}}). \quad (3)$$

Each individual waypoint sequence $w_i = (w_{i,1}, w_{i,2}, \dots)$ is an ordered list of coordinates that defines the mission path for UAV u_i . The core challenge for f_{plan} is to ground

the abstract subtasks from Π_{assigned} into meaningful and accurate spatial coordinates.

The mechanism to achieve this involves an innovative architecture where the LLM acts as a semantic-to-spatial coordinator. Instead of generating executable code, it translates an abstract subtask from the validated plan (e.g. `Go_to_warehouse`, `Scout_Lake_Area`) into a specific coordinate vector. The core of this approach is the rich contextual information provided to the LLM via a detailed system prompt. Critically, this translation is not a simple keyword match; it often requires the LLM to employ commonsense reasoning to infer the most appropriate spatial target. For instance, a subtask like `Scout_Lake_Area` is underspecified. The planner must infer from the prompt that the goal is to navigate to the center of the lake to begin a search pattern, thereby selecting the correct coordinates from the provided environmental data. The following subsections detail the two key components of this implementation: the design of the prompt that guides the LLM, and the mechanism for grounding abstract subtasks to navigational waypoints.

The core function of the LLM-as-Planner is to bridge the gap between symbolic task descriptions and concrete spatial targets. This is achieved by leveraging the LLM's ability to perform contextual information extraction from

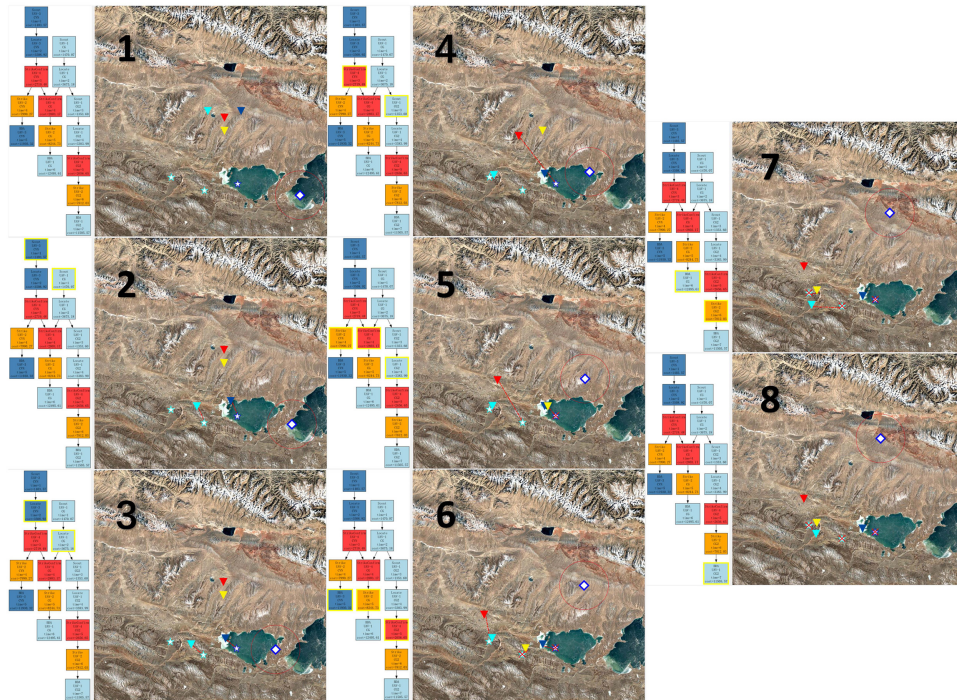


Fig. 2. The subtask assignment result and execution flow obtained by HTN after acquiring the human natural language instructions translated by LLM-as-Translator.

the system prompt. For each subtask τ_i in the validated plan H_{valid} , the LLM is prompted to generate its corresponding navigational waypoint w_i (Table 2).

The process relies on the rich environmental information embedded in the prompt, which provides a mapping of semantic names to physical coordinates (e.g. storehouse: $[x_s, y_s, z_s]$, target_A: $[x_A, y_A, z_A]$). When processing a subtask like `Go_to_storehouse`, the LLM identifies the keyword “storehouse”, searches the provided context to retrieve its associated coordinates, and outputs this vector as the waypoint. This transforms the LLM into a semantic query engine that operates over a structured textual knowledge base. For example, the abstract task “UAV₂ strikes *target_A*” is not translated into a function call, but is instead grounded to the specific coordinate vector for Target_A, which is then passed to the downstream motion planner as the mission objective. This approach ensures that all generated waypoints are directly tied to the verified world model provided in the prompt, enhancing the reliability of the generated spatial targets.

We integrate a series of path planning and trajectory tracking algorithms, including RRT,²⁹ artificial potential field method,³⁰ and carrot-chasing.³¹ These algorithms form the motion planning layer of our framework. They are not directly called by the LLM. Instead, they take the key waypoints generated by the LLM-as-Planner as input to compute the final, collision-free trajectories for the UAVs.

Table 2. System prompt for LLM-as-Planner.

Role Definition
You are a waypoint generation assistant. Your task is to parse a time-ordered list of subtask assignments for multiple UAVs and generate a structured JSON object containing the navigational waypoint sequence for each vehicle.
Input Format Specification
You will receive two pieces of information:
Initial Positions: A JSON object mapping each vehicle_id to its starting coordinates at time=0.
“UAV_1”: {“x”: 0, “y”: 0, “z”: 10, ...}
Task Assignment Log: A list of assignments, formatted as:
Time: {t} Vehicle: {UAV_ID} Target: {Target_Name}({X,Y}) Task: {Task_Type}
Waypoint Extraction Rules
1: Process the log chronologically for each UAV.
2: Extract the target coordinates (X,Y) from the “Target” .
3: Path Simplification: If a target is assigned multiple consecutive times, only retain the first occurrence.
Critical Instructions
1: Ensure all coordinate values in the output are numeric types, not strings.
2: Do not include the “algorithm_params” key in the output.
3: Strictly adhere to the specified input and output formats.

4.4. Validation

To ensure the robustness and safety of the entire system, we introduce a comprehensive validation architecture that addresses potential errors at both the task and motion planning stages.

The primary validation mechanism, Strategic Task Deconfliction, operates on the high-level task plan, H_{assigned} , generated by the HTN planner. This is a deterministic, rule-based validator designed to resolve logical and resource-based conflicts that may arise from the initial planning phase. The validation function, denoted as $V_{\text{strategic}}$, systematically inspects the plan for potential issues. A plan H_{assigned} is deemed valid only after it has been processed to resolve all identified conflicts. This can be formally expressed as

$$H_{\text{valid}} = V_{\text{strategic}}(H_{\text{assigned}}). \quad (4)$$

Here, $V_{\text{strategic}}$ implements a set of predefined rules to enforce temporal consistency — ensuring tasks adhere to causal dependencies and resource arbitration, preventing the assignment of conflicting tasks to a single UAV. The semantic coherence of the plan with the original user instruction I is inherently maintained, as the upstream LLM-as-Translator has already grounded the command in the structured vocabulary required by the planner.

The second validation component serves as a Grounding-Level Sanity Check. This check is applied after the LLM-as-Planner converts the validated abstract plan H_{valid} into a sequence of concrete navigational waypoints $W = \{w_1, w_2, \dots, w_N\}$. This lightweight, rule-based filter is crucial for mitigating risks associated with potential LLM hallucinations. Its function, $V_{\text{grounding}}$, verifies the physical plausibility of each individual waypoint w_i before it is included in the final set of executable waypoints. A waypoint is considered valid if and only if it satisfies a set of deterministic rules, which can be formally represented as

$$V_{\text{ground}}(w_i) = (w_i \in A_{\text{op}}) \wedge (\text{dis}(w_i, \text{pos}(O_i)) \leq d_{\text{max}}), \quad (5)$$

where $w_i \in A_{\text{op}}$ confirms that the waypoint lies within the predefined operational area A_{op} , which contains all mission-relevant locations and excludes no-fly zones. The second term checks that the Euclidean distance between the waypoint w_i and the known position of its corresponding task objective O_i does not exceed a maximum allowable threshold d_{max} . This ensures that the LLM’s generated spatial target is directly relevant to the task it is intended to accomplish. Only waypoints satisfying this condition are passed to the downstream motion planner,

providing a robust and safe bridge between high-level tasking and real-world motion execution.

5. Experiments

5.1. Simulation environment setup

We conduct all experiments in AirSim, a high-fidelity simulator built on Unreal Engine 4. Our simulation environment is a 5 km × 5 km outdoor scene, featuring various structures such as trees, a lake, and roads, which act as static obstacles. The environment contains 5–10 potential targets and points of interest. The multi-UAV system consists of eight quadrotor UAVs, which are heterogeneous in their onboard capabilities (e.g. some equipped with cameras for reconnaissance, others for simulated strikes), requiring the planner to perform capability-aware task assignment.

5.2. Task design for evaluation

We designed four tasks with a progressive increase in complexity to systematically evaluate our framework, as detailed in Table 3. This task set is specifically designed to test the core facets of task planning.

- **T1 (Sequential Patrol)** establishes a baseline by testing the framework's ability to generate a correct action sequence for a single UAV, which is the most fundamental planning capability.
- **T2 (Parallel Patrol)** introduces the challenge of multi-UAV coordination, requiring the system to correctly parse commands involving simultaneous, independent actions for multiple UAVs.
- **T3 (Sequential Coordination)** elevates the complexity by introducing a causal dependency between UAVs, which directly tests the framework's ability to reason about and enforce temporal and causal constraints in a multi-UAV context.
- **T4 (Conditional Coordination)** directly assesses a key contribution of our work: the ability to interpret and execute conditional logic from natural language, a scenario that is often intractable for purely formal planners.

We quantify the difficulty of each task using a complexity score S_C , calculated based on the number of UAVs, subtasks, and logical dependencies, defined as

$$S_C = N_u + N_t + 2 \cdot N_d, \quad (6)$$

where N_u is the number of UAVs, N_t is the total number of subtasks across all UAVs, and N_d is the number of dependencies or conditional branches between UAVs. The higher weight on N_d reflects the increased reasoning load for handling complex logic. This scoring allows for an objective comparison of task complexities, as shown in Table 3.

5.3. Metrics

Although we have included validation sessions throughout the task planning process, there is no guarantee that the generated scenarios will be executed correctly and without errors.

To evaluate the superiority of our proposed method, we propose three metrics to evaluate the proposed method, referring to¹⁵:

- (1) **Task Success Rate (%SR)↑**: This is the primary metric evaluating the overall effectiveness of the framework. A task is considered successful ($S_n = 1$) only if the generated plan Π is fully executed and achieves the goal described in the natural language instruction I . For unsuccessful completion, we manually assign a partial score ($0 < S_n < 1$) based on the degree of completion. The final score is the average over N_{epi} trials.

$$SR = \frac{1}{N_{\text{epi}}} \sum_{n=1}^{N_{\text{epi}}} S_n. \quad (7)$$

- (2) **Plan Robustness (%PB)↑**: This metric evaluates the system's resilience to external variations in natural language commands. For each task, we create a set of K semantically equivalent but phrased differently instructions $\{I_1, I_2, \dots, I_K\}$. We measure the success rate SR for each instruction. The robustness score is defined as the minimum success rate observed among all phrasings:

Table 3. Task design for evaluation.

Task #ID	Task name	Key challenge tested	Complexity score*
T1	Sequential patrol	Group command and single-UAV planning.	4–5
T2	Parallel patrol	Subgrouping and parallel planning.	8–10
T3	Sequential coordination	Chained inter-UAV dependencies.	13–15
T4	Conditional coordination	Conditional logic and role-based allocation.	18–20+

$$PR_i = \min k \in 1, \dots, KSR_{i,k}. \quad (8)$$

- (3) *Planning Consistency (%PC)↑*: This metric evaluates the internal reliability and predictability of the planning system. For a single, fixed natural language instruction I , we run the system M times to generate a set of plans $\{\Pi_1, \Pi_2, \dots, \Pi_M\}$. The score measures the average structural similarity among the generated plans, calculated using the normalized Levenshtein distance similarity:

$$PC = \frac{2}{M(M-1)} \sum_{1 \leq i < j \leq M} \left(1 - \frac{\text{Levenshtein}(W_i, W_j)}{\max(|W_i|, |W_j|)} \right). \quad (9)$$

We normalized the above metrics to facilitate analysis and comparison between different features.

5.4. Baseline method

To rigorously evaluate the contributions of our framework's components, we compare MUTP-LLM against two baseline methods that represent different approaches to LLM-driven planning.

- **Rule-based Planner** serves as a non-LLM baseline. It consists solely of a Hierarchical Task Network (HTN) planner. To interact with it, a human expert must manually translate the natural language instruction into a formal, pre-defined task representation (e.g. a structured file) that the HTN can understand.
- **LLM-to-PDDL** represents a popular hybrid approach where an LLM is used to translate natural language instructions into the Planning Domain Definition Language (PDDL). A classical PDDL planner then generates the task plan. While this leverages formal planning guarantees, it is susceptible to context collapse, where nuanced but critical information from

the original command can be lost during the translation to the rigid symbolic representation of PDDL.

- **Direct-LLM-Planner** represents a distinct and influential paradigm where the LLM functions as a zero-shot, end-to-end task planner. Inspired by prior works such as Inner Monologue¹⁸ and ProgPrompt,²⁸ this approach tasks the LLM with directly translating the high-level natural language instruction into a complete sequence of executable API calls in a single pass. In this setup, the entire planning process is performed implicitly within the LLM's internal reasoning process. It does not rely on an external, structured planner like HTN or formal symbolic representations.

6. Results

6.1. Overall performance

Table 4 presents the comprehensive results of task planning across the four types of tasks. The data clearly show that MUTP-LLM consistently outperforms all baseline methods across all tasks and metrics, demonstrating its superior effectiveness and robustness, especially as task complexity increases.

Specifically, MUTP-LLM is the only framework that maintains a high Task Success Rate (SR) across the entire range of task complexities, achieving 87% on the most difficult Conditional Coordination task (T4). In contrast, the baseline methods reveal critical weaknesses. The Rule-based Planner, for instance, proves fragile; its success is confined to simple tasks and collapses when faced with linguistic nuances (SR drops to 0% for T4) due to its rigid keyword-matching approach. The LLM-to-PDDL planner, while more flexible, also falters on complex tasks (SR 45% on T4), primarily due to context collapse, where crucial conditional information is lost during the translation to the rigid PDDL format. Finally, the Direct-LLM-Planner often generates logically incoherent plans for coordinated missions, as it lacks an external reasoning

Table 4. Overall task planning performance.

Method (DeepSeek-V3)	Task #ID											
	Task 1			Task 2			Task 3			Task 4		
	SR (%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)
Rule-based planner	100	94	100	98	47	95	40	15	94	0	0	N/A
LLM-to-PDDL	99	98	75.83	78	93	51.45	80	74	58.05	45	31	47.76
MUTP-LLM	100	100	99.2	98	96	91.7	99	98	73.9	87	76	71.4
Models	Task 1			Task 2			Task 3			Task 4		
	SR (%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)
	SR (%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)	SR(%)	PB(%)	PC(%)
Qwen-2.5-32B	100	100	99.89	96	94	94.4	100	97	81.2	91	77	73.98
GLM-4	96	91	95.44	93	87	94.99	93	89	75.0	84	69	95.0

scaffold. A more detailed analysis of this paradigm's limitations is presented in our ablation studies.

Analyzing the performance of different LLMs within our MUTP-LLM framework, both DeepSeek-V3 and Qwen-2.5-32B achieve excellent results, with GLM-4 being slightly less performant. The primary reason for GLM-4's slightly lower scores is its tendency to generate outputs that do not strictly adhere to the requested JSON format. These formatting errors, while semantically minor, lead to parsing failures, underscoring the necessity for models to maintain strict structural compliance in autonomous systems.

Furthermore, it is important to analyze the Planning Consistency (PC) metric. The observed decrease in PC for Task 4 (e.g. 71.4% for MUTP-LLM with DeepSeek) does not indicate a flaw in planning. Instead, this is an expected outcome because T4 involves multiple UAVs with identical capabilities (e.g. two reconnaissance UAVs). The planner can validly assign a task to either of these UAVs, resulting in plans that are functionally identical and correct but differ in their textual representation.

6.2. Ablation studies

We conduct an ablation study to analyze the effectiveness of each component within the MUTP-LLM framework. We maintain consistent hyper-parameters across experiments to isolate the effects of these factors. By systematically removing key modules and observing the performance degradation, we can quantify their importance. The results are summarized in Table 5.

Importance of LLM-as-Translator. In this ablation, we removed the LLM-as-Translator, requiring manual, structured task input to the HTN planner. As shown in Table 5, this configuration's Plan Robustness (PB) plummets from 76% to a mere 35%. Although its Task Success Rate (SR) remains high for correctly formatted inputs, its inability to handle linguistic variations makes it brittle. This data confirms that the LLM-as-Translator is critical for bridging the semantic gap and providing the necessary flexibility to interpret the nuanced language of real-world human commands.

Importance of HTN-Planner. To evaluate the role of the structured symbolic planner, we conducted an ablation study by removing the HTN Planner from the framework. In this configuration, the LLM-as-Translator's output (a sequence of unstructured tasks) is passed directly to the LLM-as-Planner, which must then handle the complex multi-agent task assignment and sequencing

implicitly. As shown in Table 5, this change caused a catastrophic drop in Planning Consistency (PC) from 71.4% to a mere 27.83%, the lowest of any ablation. While the Task Success Rate (SR) only moderately decreased to 75%, the collapse in PC is highly revealing. It demonstrates that the HTN Planner serves as an indispensable reasoning scaffold. It offloads the burden of complex combinatorial logic from the LLM, ensuring that the resulting plans are not only successful but also logically coherent, consistent, and well-structured.

Importance of Validation. Finally, we removed the two-stage validation module. This configuration proved to be dangerously unreliable. According to Table 5, without validation, the SR drops from 87% to 64% and PB falls from 76% to 53%. We observed two primary failure modes: (1) logical conflicts (e.g. redundant UAV allocation) were not caught, and (2) physically unsafe waypoints (e.g. inside obstacles) hallucinated by the LLM-as-Planner were not filtered. This result provides clear quantitative evidence that the validation architecture is a non-negotiable safety layer for any real-world deployment.

The critical role of a structured framework is further underscored by the performance of the Direct-LLM-Planner baseline in Table 5. With an SR of only 15% and a PB of 3%, this approach fails almost completely. Even with more detailed prompts, its performance only marginally improves to an SR of 21%. This demonstrates that relying solely on the prompt to guide an LLM for complex planning is insufficient and that a hybrid, structured architecture like MUTP-LLM is indispensable for achieving robust and reliable performance.

6.3. Qualitative analysis

In Fig. 3, we visualize the step-by-step execution of MUTP-LLM, translating a complex natural language command into a detailed symbolic task schedule. This plan is then grounded into navigational waypoints, which are subsequently coordinated in time to produce a deconflicted, multi-UAV trajectory plan. The resulting

Table 5. Results of ablation studies.

Method	Metrics		
	SR%	PB%	PC%
MUTP-LLM	87	76	71.4
w/o LLM-as-Translator	39	35	68.15
w/o HTN Planner	75	72	27.83
w/o Validation	64	53	70.98
Direct-LLM-Planner	15	3	12.6
Direct-LLM-Planner (with more prompts)	21	7	17.49

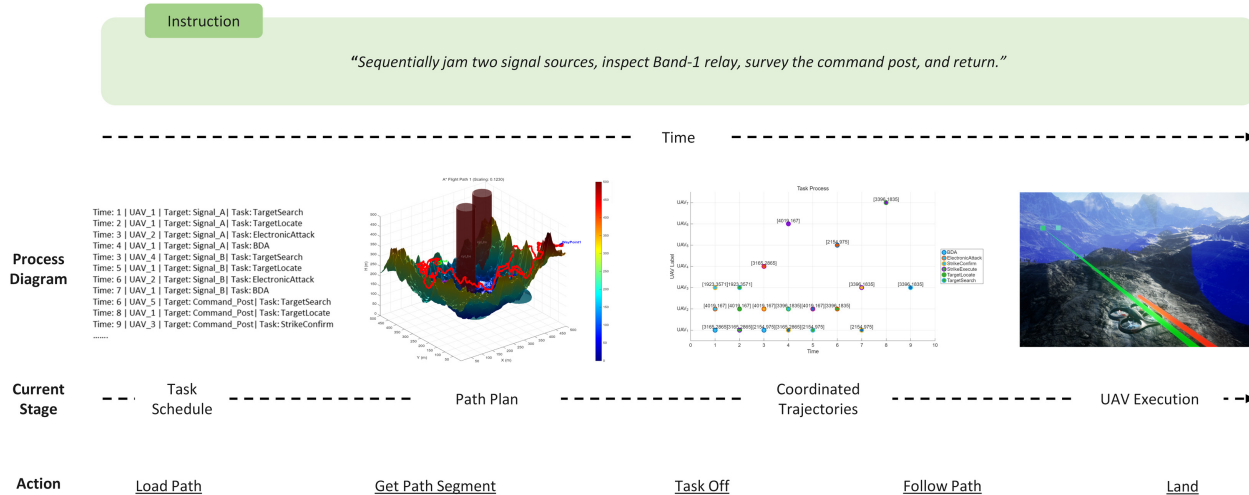


Fig. 3. Qualitative example of our MUTP-LLM performing a multi-UAV mission, visualizing the process from natural language instruction to coordinated, multi-UAV execution in simulation.

plan's successful execution is shown in the AirSim simulator, highlighting how our framework synergizes LLM-based semantic interpretation with formal planners for robust autonomy. This seamless process validates our method's ability to handle intricate, real-world mission scenarios from high-level human intent.

7. Conclusions and Future Works

This paper introduced MUTP-LLM, a novel hybrid framework that successfully addresses the challenge of multi-UAV task planning from natural language commands. By synergizing LLM with HTN planner, our architecture effectively bridges the gap between high-level human intent and low-level executable actions. The framework's strength lies in its hierarchical structure, featuring an LLM-as-Translator for semantic interpretation, an LLM-as-Planner for contextual grounding, and a two-stage validation mechanism that ensures both logical coherence and physical safety.

Future work will transition from the current offline paradigm to online replanning, enabling the system to dynamically adjust strategies using real-time feedback. Beyond ensuring physical safety, we will also address the crucial challenge of semantic hallucination by developing a dedicated critic LLM to verify that generated plans are not only plausible but also contextually sound and trustworthy.

In summary, MUTP-LLM represents a significant step towards creating reliable and intelligent autonomous systems. The principles of hybrid architectures, which balance learning-based flexibility with formal guarantees, will continue to be a cornerstone of our future work.

By tackling the challenges of online adaptation and semantic verification, we can move closer to deploying a robust autonomous multi-UAV system in the complexities of the real world.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant Nos. 62576349 and 61876187.

ORCID

Hongbo Yu <https://orcid.org/0009-0003-5054-839X>
 Chang Wang <https://orcid.org/0000-0003-0161-0591>
 Yifeng Niu <https://orcid.org/0009-0001-5847-2575>
 Lizhen Wu <https://orcid.org/0000-0002-0452-2602>

References

1. C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling and T. Lozano-Pérez, Integrated Task and Motion Planning, *Annu. Rev. Control Robot. Auton. Syst.* **4**(1), 265–293 (2021).
2. M. Fox and D. Long, PDDL2. 1: An Extension to PDDL for Expressing Temporal Planning Domains, *J. Artif. Intell. Res.* **20**, 61–124 (2003).
3. C. R. Garrett, T. Lozano-Pérez and L. P. Kaelbling, PDDLStream: Integrating Symbolic Planners and Black-box Samplers via Optimistic Adaptive Planning, in *Proc. Int. Conf. Automated Planning and Scheduling*, Vol. 30 (Association for the Advancement of Artificial Intelligence, 2020), pp. 440–448.
4. P. Hashem, N. Lipovetzky, D. Magazzeni, C. Muise, R. Brachman, F. Rossi and P. Stone, *An Introduction to the Planning Domain Definition Language*, Vol. 13 (Springer, 2019).

5. D. Sun, J. Chen, S. Mitra and C. Fan, Multi-Agent Motion Planning from Signal Temporal Logic Specifications, *IEEE Robot. Autom. Lett.* **7**, 3451–3458 (2022).
6. J. Pan, G. Chou and D. Berenson, Data-Efficient Learning of Natural Language to Linear Temporal Logic Translators for Robot Task Specification, in *2023 IEEE Int. Conf. Robotics and Automation (ICRA)* (IEEE, 2023), pp. 11554–11561.
7. J. Thomason, A. Padmakumar, J. Sinapov, N. Walker, Y. Jiang, H. Yedidsion, J. Hart, P. Stone and R. Mooney, Jointly Improving Parsing and Perception for Natural Language Commands through Human-Robot Dialog, *J. Artif. Intell. Res.* **67**, 327–374 (2020).
8. C. Wang, Z. Zhong, X. Xiang, Y. Zhu, L. Wu, D. Yin and J. Li, UAV Path Planning in Multi-Task Environments with Risks through Natural Language Understanding, *Drones* **7**(3), 147 (2023).
9. H. Liu, Q. Chen, N. Pan, Y. Sun, Y. An and D. Pan, UAV Stocktaking Task-Planning for Industrial Warehouses Based on the Improved Hybrid Differential Evolution Algorithm, *IEEE Trans. Ind. Inform.* **18**(1), 582–591 (2021).
10. T. Silver, R. Chitnis, A. Curtis, J. B. Tenenbaum, T. Lozano-Pérez and L. P. Kaelbling, Planning with Learned Object Importance in Large Problem Instances Using Graph Neural Networks, in *Proc. AAAI Conf. Artificial Intelligence*, Vol. 35 (AAAI, 2021), pp. 11962–11971.
11. A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, DeepSeek-V3 Technical Report, preprint (2024), arXiv:2412.19437.
12. J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, Gpt-4 Technical Report, preprint (2023), arXiv:2303.08774.
13. S. H. Vempala, R. Bonatti, A. Bucker and A. Kapoor, Chatgpt for Robotics: Design Principles and Model Abilities, *IEEE Access* **12**, 55682–55696 (2024).
14. M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman *et al.*, Do as I Can, Not as I Say: Grounding Language in Robotic Affordances, preprint (2022), arXiv:2204.01691.
15. W. Huang, P. Abbeel, D. Pathak and I. Mordatch, Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents, in *Int. Conf. Machine Learning* (PMLR, 2022), pp. 9118–9147.
16. K. Valmeekam, A. Olmo, S. Sreedharan and S. Kambhampati, Large Language Models Still Can't Plan (a Benchmark for LLMs on Planning and Reasoning About Change), preprint (2022), arXiv:2206.10498.
17. Y. Jin, D. Li, A. Yong, J. Shi, P. Hao, F. Sun, J. Zhang and B. Fang, Robotgpt: Robot Manipulation Learning from Chatgpt, *IEEE Robot. Autom. Lett.* **9**, 2543–2550 (2024).
18. W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, Inner Monologue: Embodied Reasoning through Planning with Language Models, preprint (2022), arXiv:2207.05608.
19. M. Skreta, N. Yoshikawa, S. Arellano-Rubach, Z. Ji, L. Bjørn Kristensen, K. Darvish, A. Aspuru-Guzik, F. Shkurti and A. Garg, Errors Are Useful Prompts: Instruction Guided Task Programming with Verifier-Assisted Iterative Prompting, preprint (2023), arXiv:2303.14100.
20. R. Lallement, L. De Silva and R. Alami, HATP: An HTN Planner for Robotics, preprint (2014), arXiv:1405.5345.
21. D. Höller, G. Behnke, P. Bercher, S. Biundo, H. Fiorino, D. Pellier and R. Alford, HDDL: An Extension to PDDL for Expressing Hierarchical Planning Problems, in *Proc. AAAI Conf. Artificial Intelligence*, Vol. 34 (AAAI, 2020), pp. 9883–9891.
22. D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine and K. Hausman, Mtpopt: Continuous Multi-Task Robotic Reinforcement Learning at Scale, preprint (2021), arXiv:2104.08212.
23. B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas and P. Stone, LLM+ P: Empowering Large Language Models with Optimal Planning Proficiency, preprint (2023), arXiv:2304.11477.
24. T. Silver, V. Hariprasad, R. S. Shuttlesworth, N. Kumar, T. Lozano-Pérez and L. P. Kaelbling, PDDL Planning with Pretrained Large Language Models, in *NeurIPS 2022 Foundation Models for Decision Making Workshop* (NeurIPS, 2022), pp. 1–13.
25. Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong and H. Soh, Translating Natural Language to Planning Goals with Large-Language Models, preprint (2023), arXiv:2302.05128.
26. C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao and Y. Su, LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models, in *Proc. IEEE/CVF Int. Conf. Computer Vision* (IEEE, 2023), pp. 2998–3009.
27. J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence and A. Zeng, Code as Policies: Language Model Programs for Embodied Control, in *2023 IEEE Int. Conf. Robotics and Automation (ICRA)* (IEEE, 2023), pp. 9493–9500.
28. I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason and A. Garg, ProgPrompt: Generating Situated Robot Task Plans Using Large Language Models, preprint (2022), arXiv:2209.11302.
29. S. LaValle, Rapidly-Exploring Random Trees: A New Tool for Path Planning, Research Report 9811, Department of Computer Science, Iowa State University (1998).
30. O. Khatib, Real-Time Obstacle Avoidance for Manipulators and Mobile Robots, *Int. J. Robot. Res.* **5**, 90–98 (1996).
31. R. Bhadani, Path Planning of Unmanned System Using Carrot-Chasing Algorithm, preprint (2020), arXiv:2012.13227.



Hongbo Yu received his B.E. degree from the National University of Defense Technology (NUDT), Changsha, China, in 2022. He is currently working toward the Ph.D. degree in Control Science and Engineering with the College of Intelligence Science and Technology, National University of Defense Technology, Changsha, China. His current research interests include artificial intelligence, autonomous control and embodied intelligence.



Chang Wang received his B.S. degree in Mathematics from the University of Science and Technology of China, Hefei, China, in 2003, the M.Sc. degree in Applied Mathematics from the National University of Defense Technology (NUDT), Changsha, China, in 2009, and the Ph.D. degree in Robot Learning from Delft University of Technology, Delft, The Netherlands, in 2016. He is currently an Associate Professor with the College of Intelligence Science and Technology, NUDT. His research interests include reinforcement learning, developmental robotics, and human-robot teamwork.



Yifeng Niu received his B.S. and Ph.D. degrees from the National University of Defense Technology (NUDT), China, in 2001 and 2007, respectively. He was a visiting scholar in the Department of Computer Science of Tuebingen University in Germany in 2012. He is now a Professor at the College of Intelligence Science and Technology in NUDT. His current research interests include autonomous control, cooperative control, and environment perception and cognition. He is the corresponding author of this paper.



Lizhen Wu is an Assistant Professor at the College of Intelligence Science and Technology, National University of Defense Technology, China. He received his B.E. degree in Mechanical Engineering and Automation from Xian Jiaotong University in 2004, and his Ph.D. in Control Science and Engineering from National University of Defense Technology in 2013. His main research interests include unmanned system, pattern recognition, and autonomous control theory and applications.